

05 — CONFUSION TABLE (the pairs students mix up)

Each block: the one-sentence difference, then the side-by-side table.

1. SQA vs SQC

Difference: SQA **prevents** defects by managing the process (never runs the program); SQC **finds** defects by evaluating the product (always runs the program).

Aspect	SQA	SQC
Aim	Prevent defects	Identify & resolve defects
Runs the program?	No	Yes
Orientation	Process-oriented	Product-oriented
Maps to	Verification	Validation
Responsibility	All team members	Only testing team
System type	Failure prevention	Failure detection
Scope	Full SDLC	Testing life cycle

2. Error vs Fault vs Failure vs Defect

Difference: an **error** (human) injects a **fault** (in the code), which when executed causes a **failure** (wrong behaviour); all three are collectively called **defects** ("bugs").

Term	What it is	Where
Error	Human action producing an incorrect result	In a person
Fault	Incorrect step/process/data definition	In the program
Failure	Inability to perform the required function	At run time
Defect	Behavioural deviation from spec (umbrella term)	Anywhere

3. Verification vs Validation

Difference: verification asks "are we building it **right**?" (process); validation asks "are we building the **right** thing?" (product).

Verification	Validation
Doing the things right	Results are what you expected
Process-oriented	Product-oriented
Linked to SQA	Linked to SQC
Reviews, inspections	Testing, user acceptance

4. Static vs Dynamic analysis

Difference: static analysis reads the code/documents **without running** it; dynamic analysis **executes** the program to see failures.

Static (manual)	Dynamic (automated)
No execution	Actual execution
Reasons over all possible behaviours	Observes representative behaviour
Code review, inspection, algorithm analysis	Running test cases
Slow to review	Finds only what runs; "can't think outside the box"

5. Quality Factor vs Quality Criterion

Difference: a **factor** is an external behavioural characteristic of the system; a **criterion** is an internal attribute of a factor tied to development.

Quality Factor	Quality Criterion
Behavioural characteristic of the system	Attribute of a quality factor
External view	Development-related
Reliability, Correctness, Efficiency, Performance, Usability, Integrity	Modularity, Testability, Maintainability, Reusability, Flexibility, Portability
From McCall's model	From McCall's model

Trap: Reliability = factor; Modularity = criterion.

6. White-box vs Black-box vs Gray-box

Difference: white-box tests the **inside** (code), black-box tests the **outside** (behaviour), gray-box **mixes** both at the middle level.

	White-box	Black-box	Gray-box
Also called	Structural / implementation-based	Functional / specification-based	Mixed
Sees	Source code (control & data flow)	External interface only	Both, partially
Object size	Small units	Large systems	Middle (procedures in modules)
Timeline	Early (unit/component)	Late (system/acceptance)	Middle
Tester	Developers	Professional / 3rd-party (IV&V)	—

7. CMM vs TMM (levels side by side)

Difference: CMM measures the **development** process maturity; TMM measures the **testing** process maturity (by Ilene Burnstein).

Level	CMM (development)	TMM (testing)
1	Initial	Initial
2	Repeatable	Phase Definition
3	Defined	Integration
4	Managed	Management and Measurement
5	Optimizing	Optimization / Defect Prevention & QC

Trap: CMM 4 = Managed, 5 = Optimizing (don't reverse).

8. Test Plan vs Test Case vs Test Suite

Difference: the **plan** is the high-level strategy, a **case** is one low-level input+expected-result, and a **suite** is the whole set of cases run together.

	Test Plan	Test Case	Test Suite
Level	High-level	Low-level	Macro-level
Content	Objectives, scope, approach, resources, schedule	One input/action + expected result	Collection of test cases
Purpose	How testing will be done	Check one feature	Run until stopping criteria

9. Manual vs Automated testing

Difference: manual testing is human-run (rigorous but slow); automated testing is tool-run (fast but can't judge usability) — and automation **can't fully replace** manual.

Manual	Automated
Oldest, most rigorous	Uses software tools
Tester performs each step	Runs without manual intervention
Hard to repeat, costly, slow, labour-intensive	Fast, repeatable, reliable, reusable, programmable
Needed for: usability, logical errors, ad-hoc, one-time	Good for: regression, data-driven, load/stress, GUI internals

10. Defect Prevention vs Reduction vs Containment

Difference: prevention stops faults being **injected**, reduction **removes** injected faults, containment **limits the damage** of the failures that remain.

	Prevention	Reduction	Containment
Acts on	Errors (before injection)	Faults (after injection)	Failures (at run time)
Goal	Don't inject the fault	Detect & remove the fault	Contain/limit the failure
Techniques	Error source removal, error blocking	Inspection, testing	Fault tolerance (NVP, recovery), safety assurance
Example	Default values, data validation	Code inspection	Auto-save, exception handling

Trap: NVP & auto-save = **containment**, not prevention. Prevention is never 100%.

11. Internal vs External view

Difference: the external/consumer view sees a **black box** (behaviour only); the internal/producer view sees a **white box** (the workings).

External / Consumer	Internal / Producer
Customers and users	Developers, testers, managers
Black box	White / clear box
"Good enough" for the price	"Good enough" for the cost
Functionality, reliability, usability, safety	Maintainability, interoperability, modularity

12. Developer vs Tester role

Difference: the developer **builds** the product (constructive, driven by delivery); the tester tries to **break** it (destructive, driven by quality).

Developer	Tester
Implements requirements, designs, programs	Plans testing, designs test cases
Success = creating a product	Success = finding errors
Perceptions constructive	Perceptions destructive
Software is known; driven by delivery	Software is unknown; driven by quality
Debugging & correcting defects	Test & re-test

13. Vertical vs Horizontal vs Mixed testing team

Difference: vertical = many testing tasks for **one product**; horizontal = **one testing type** across many products; mixed = both.

Vertical	Horizontal	Mixed
Organized around one product	Organized around one testing type	Combines both
Dedicated people do several tasks for that product	Do one kind of test for many products	Used by large organizations